

Testing Exceptions in JUnit



Some **methods** are expected to **throw exceptions** under **invalid conditions**. In the example discussed in this lesson, **integer division by zero** should trigger an **ArithmeticException**. Using clear **Arrange-Act-Assert** sections improves understanding of **exception tests**. **Lifecycle methods** can supply **shared objects** (e.g., the `calculator` **instance**).

 by Rodrigo Martins Pagliares

Last update: 11/27/2025

Previously in this course...

```
68 // @Disabled("TODO: Further work is required here.")
69 @DisplayName("Trying to divide by zero")
70 @Test
71 ▶ public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {
72     System.out.println("Running division by zero");
73     // fail("Not yet implemented");
74 }
75 }
```

Since we had not discussed how to assert an exception, we decided to disable this test method in a previous lesson. Time to tackle it, by first commenting out the `@Disabled` annotation and the `fail` test method!



Using assertThrows



- JUnit provides **assertThrows** to verify that a specific exception type is thrown
- **First argument**: the expected exception class (`ArithmeticException.class`)
- **Second argument**: a lambda expression invoking the method under test
- **Optional**: include a custom failure message to clarify expectations

Inspecting the Exception Message



Return Value

`assertThrows` returns the actual exception object for further inspection

Store & Check

Store it in a **local variable** to check details, such as the **message text**

Compare Messages

Compare **expected** and **actual messages** using `assertEquals`

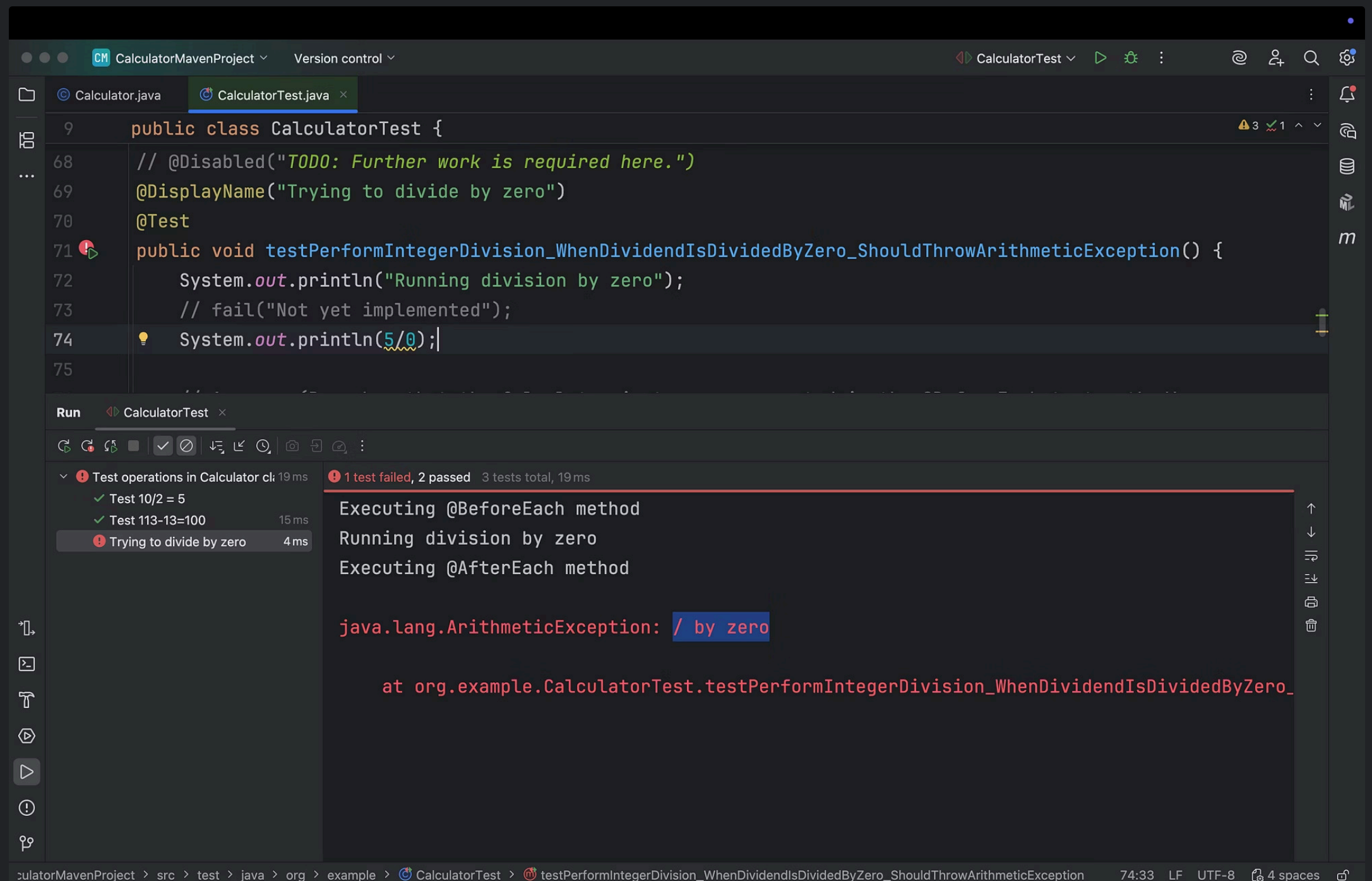
Validation

Helps validate both the type and the content of the **thrown exception**

Message shown when an ArithmeticException is Thrown

Let's include a error logic in our code to see the exactly error message shown in the console when an ArithmeticException is thrown:

```
System.out.println(5/0);
```



The screenshot shows an IDE window with a Java test class `CalculatorTest.java` and its execution results. The code in the editor is as follows:

```
9 public class CalculatorTest {
68 // @Disabled("TODO: Further work is required here.")
69 @DisplayName("Trying to divide by zero")
70 @Test
71 public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {
72     System.out.println("Running division by zero");
73     // fail("Not yet implemented");
74     System.out.println(5/0);
75 }
```

The Run tab shows the execution results for the test `testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException`. The test failed, and the error message is displayed in the console:

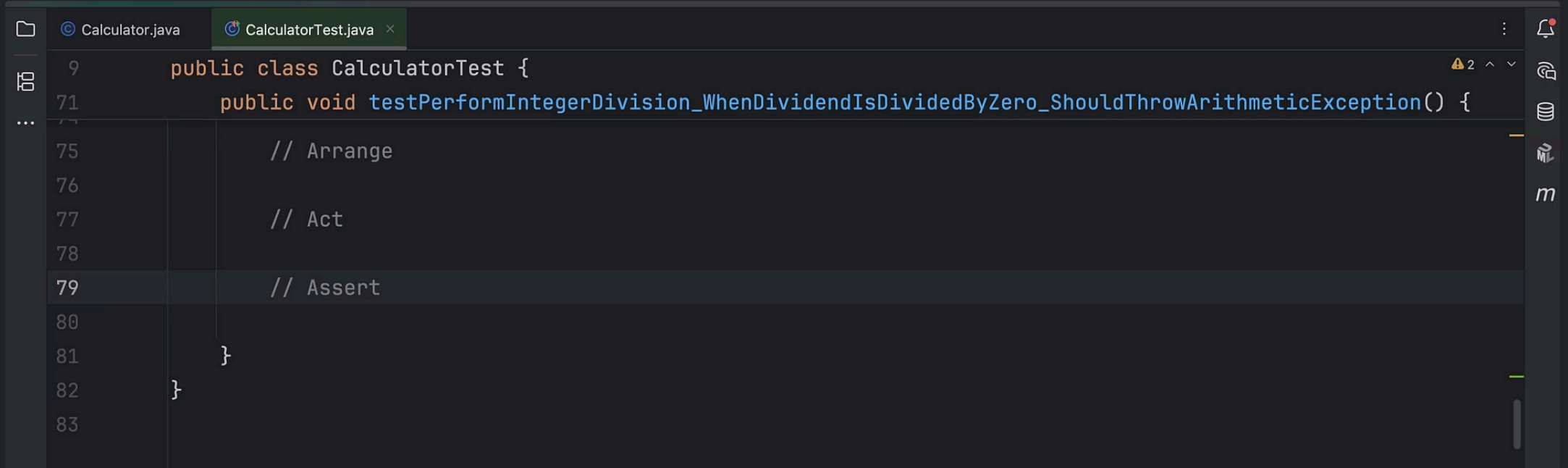
```
Executing @BeforeEach method
Running division by zero
Executing @AfterEach method

java.lang.ArithmeticException: / by zero

at org.example.CalculatorTest.testPerformIntegerDivision_WhenDividendIsDividedByZero_
```

Notice that the message is `"/ by zero"`

Arrange, Act, Assert



The screenshot shows an IDE window with two tabs: 'Calculator.java' and 'CalculatorTest.java'. The 'CalculatorTest.java' tab is active, displaying the following code:

```
9      public class CalculatorTest {  
71          public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {  
74              // Arrange  
75              // Act  
76              // Assert  
79          }  
80      }  
81  }  
82  
83
```

The code is a Java test class named 'CalculatorTest'. It contains a single test method 'testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException()'. The method body is currently empty, with comments indicating where to place 'Arrange', 'Act', and 'Assert' steps. The IDE interface includes a sidebar on the left with icons for file explorer, search, and other tools, and a sidebar on the right with icons for run, debug, and other IDE features.

Arrange

```
75 // Arrange (Remember that the Calculator instance was created in the @BeforeEach test method!  
76 int dividend = 10;  
77 int divisor = 0;  
78 String expectedExceptionMessage = "/ by zero";
```

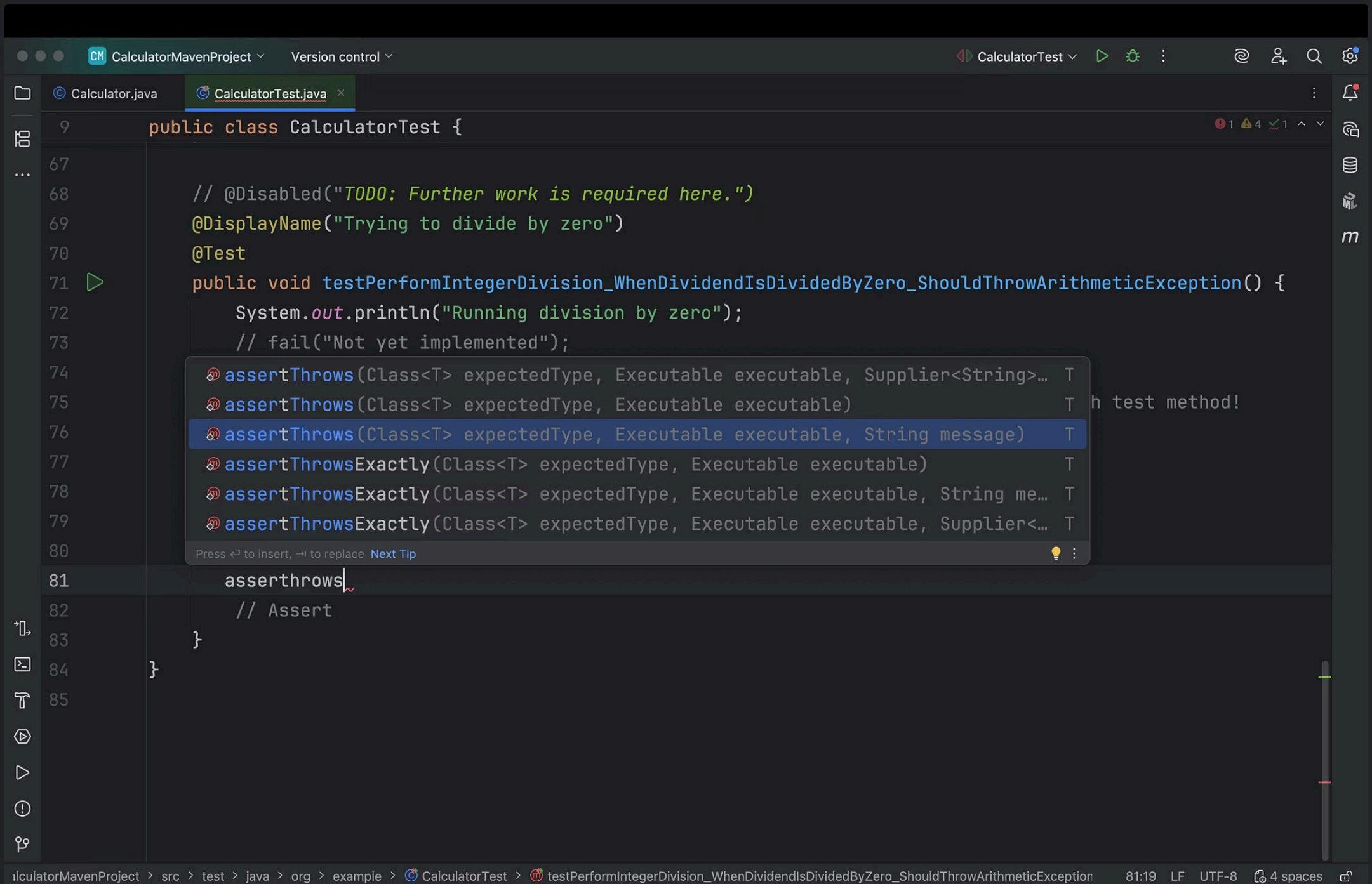
Note: I didn't know beforehand what the exact message of the `ArithmeticException` would be. That's the reason in a previous slide I deliberately included a bug to verify the message ("/ by zero").

That's the message I assigned to the variable `expectedExceptionMessage` (Line 78)

My plan is to use `expectedExceptionMessage` in the **Assert** part of **AAA**.

Act

JUnit provides **assertThrows** to verify that a **specific exception type** is thrown



The screenshot shows an IDE window for a project named "CalculatorMavenProject". The active file is "CalculatorTest.java". The code defines a class "CalculatorTest" with a test method "testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException()". The method is annotated with "@Disabled", "@DisplayName", and "@Test". It prints a message and then calls "fail()". A code completion menu is open, showing several "assertThrows" methods. The method "assertThrows(Class<T> expectedType, Executable executable, String message)" is selected. The status bar at the bottom shows the file path "calculatorMavenProject > src > test > java > org > example > CalculatorTest" and the current line is 81:19.

```
9 public class CalculatorTest {  
67  
68     // @Disabled("TODO: Further work is required here.")  
69     @DisplayName("Trying to divide by zero")  
70     @Test  
71     public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {  
72         System.out.println("Running division by zero");  
73         // fail("Not yet implemented");  
74  
75  
76  
77  
78  
79  
80  
81         assertThrows  
82         // Assert  
83     }  
84 }  
85
```

Code completion menu options:

- assertThrows(Class<T> expectedType, Executable executable, Supplier<String>... T
- assertThrows(Class<T> expectedType, Executable executable) T
- assertThrows(Class<T> expectedType, Executable executable, String message) T
- assertThrowsExactly(Class<T> expectedType, Executable executable) T
- assertThrowsExactly(Class<T> expectedType, Executable executable, String me... T
- assertThrowsExactly(Class<T> expectedType, Executable executable, Supplier<... T

Press ⌘ to insert, ⇧ to replace Next Tip

calculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException 81:19 LF UTF-8 4 spaces

Act

The `assertThrows` method wraps the call to the method under test.

```
80      // Act
81      ArithmeticException arithmeticException = assertThrows(ArithmeticException.class,
82      () -> calculator.performIntegerDivision(dividend, divisor),
83      message: "Division by zero should throw ArithmeticException");
84
```

- **First argument:** the expected exception class (`ArithmeticException.class`)
- **Second argument:** a lambda expression invoking the method under test
- **Optional:** include a custom failure message to clarify expectations

Assert

```
85         // Assert
86         assertEquals(expectedExceptionMessage, arithmeticException.getMessage(),
87             ()-> "Unexpected exception message");
88     }
```

AAA

CalculatorMavenProject

Version control

CalculatorTest

CalculatorTest

Calculator.java

CalculatorTest.java

```
9      public class CalculatorTest {
71      public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {
74
75          // Arrange (Remember that the Calculator instance was created in the @BeforeEach test method!
76          int dividend = 10;
77          int divisor = 0;
78          String expectedExceptionMessage = "/ by zero";
79
80          // Act
81          ArithmeticException arithmeticException = assertThrows(ArithmeticException.class,
82              () -> calculator.performIntegerDivision(dividend, divisor),
83              message: "Division by zero should throw ArithmeticException");
84
85          // Assert
86          assertEquals(expectedExceptionMessage, arithmeticException.getMessage(),
87              () -> "Unexpected exception message");
88      }
89  }
```

Run

CalculatorTest

Test operations in Calculator class (org.example) 15 ms

Test 10/2 = 5 13 ms

Test 113-13=100 13 ms

Trying to divide by zero 2 ms

3 tests passed 3 tests total, 15 ms

Executing @BeforeEach method

Running division by zero

Executing @AfterEach method

CalculatorMavenProject > src > test > java > org > example > CalculatorTest

87:54 LF UTF-8 4 spaces

Make the test fail

```
int divisor = 5;
```

```
9      public class CalculatorTest {
71          public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {
74
75              // Arrange (Remember that the Calculator instance was created in the @BeforeEach test method!
76              int dividend = 10;
77              💡 int divisor = 5;
78              String expectedExceptionMessage = "/ by zero";
79
80              // Act
81              ArithmeticException arithmeticException = assertThrows(ArithmeticException.class,
82                  () -> calculator.performIntegerDivision(dividend, divisor),
83                  message: "Division by zero should throw ArithmeticException");
84
85              // Assert
86              assertEquals(expectedExceptionMessage, arithmeticException.getMessage(),
87                  () -> "Unexpected exception message");
88          }
89      }
```

Make the test fail (contd.)

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The editor displays the file "CalculatorTest.java" with the following code:

```
9 public class CalculatorTest {
71     public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {
78         String expectedExceptionMessage = "/ by zero";
79
80         // Act
81         ArithmeticException arithmeticException = assertThrows(ArithmeticException.class,
82             () -> calculator.performIntegerDivision(dividend, divisor),
83             message: "Division by zero should throw ArithmeticException");
84
85         // Assert
86         assertEquals(expectedExceptionMessage, arithmeticException.getMessage(),
```

Below the editor, the "Run" tab shows the execution results for "CalculatorTest". The test suite "Test operations in Calculator class (org.example)" took 17 ms and resulted in "1 test failed, 2 passed" out of 3 tests total.

Test Name	Duration	Result
Test 10/2 = 5	1 ms	Passed
Test 113-13=100	11 ms	Passed
Trying to divide by zero	5 ms	Failed

The failed test details are shown in the right pane:

```
FailedError: Division by zero should throw ArithmeticException ==> Expected java.
>
torTest.testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArith
code 255
```

The bottom status bar indicates the file path: "calculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException", along with line 77, column 24, and encoding "UTF-8".

Make the test pass

```
int divisor = 0;
```

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The editor displays the file "CalculatorTest.java" with the following code:

```
9      public class CalculatorTest {
71      public void testPerformIntegerDivision_WhenDividendIsDividedByZero_ShouldThrowArithmeticException() {
74
75          // Arrange (Remember that the Calculator instance was created in the @BeforeEach test method!
76          int dividend = 10;
77          int divisor = 0;
78          String expectedExceptionMessage = "/ by zero";
79
80          // Act
81          ArithmeticException arithmeticException = assertThrows(ArithmeticException.class,
82              () -> calculator.performIntegerDivision(dividend, divisor),
83              message: "Division by zero should throw ArithmeticException");
84
85          // Assert
86          assertEquals(expectedExceptionMessage, arithmeticException.getMessage(),
87              () -> "Unexpected exception message");
88      }
```

Below the editor, the "Run" tab is active, showing the execution of "CalculatorTest". The test results are as follows:

Test Name	Duration	Status
Test operations in Calculator class (org.example)	15 ms	Passed
Test 10/2 = 5	12 ms	Passed
Test 113-13=100	12 ms	Passed
Trying to divide by zero	3 ms	Passed

The output console shows the following messages:

```
Executing @BeforeEach method
Running division by zero
Executing @AfterEach method
```

The status bar at the bottom indicates the file path: "CalculatorMavenProject > src > test > java > org > example > CalculatorTest". The encoding is UTF-8, and the line length is 77:25.

Conclusion



- Testing exceptions is essential for validating error-handling behavior
- **assertThrows** provides a clean and expressive way to check failure scenarios
- Messages and assertions help confirm that the method fails **as expected**
- Combining Arrange-Act-Assert and exception testing improves test clarity

Open Floor Discussion



When should you test exception messages explicitly?

What types of input validation should always include exception tests?

How do exception tests help prevent hidden bugs in real applications?